発展プログラミング (Advanced Programming)

2. デザインパターンの導入

尾崎 知伸 ＜ozaki.tomonobu@nihon-u.ac.jp＞

Template Methodパターン

～具体的な処理をサブクラスにまかせる～

Template Methodパターン

- テンプレート：定型文・ひな形・あらかじめ決められた様式

- Template Methodパターン：テンプレートの機能を持つパターン
  - 「スーパークラス」と「サブクラス」からなる非常に単純なパターン
  - スーパークラス：処理の大きな流れを定義する（処理の枠組みを定める）
  - サブクラス：各処理の具体的な処理内容を定義する

- スーパークラス側
  - テンプレレートとなるメソッドを定義する
    - そのまんま「テンプレートメソッド」と呼びます
  - テンプレートとなるメソッド内で，抽象メソッドが使われる
  - 抽象メソッドをどの様に呼び出すか（＝テンプレート）を定義

- サブクラス側
  - 抽象メソッドを実装する（細かな挙動の定義）

- 利用シーン
  - 似てはいるが，具体的な処理が異なる複数のクラスを作成したい場合

→ 個々のクラスで大枠の処理のアルゴリズムを考えなくてよい

→ 大枠の処理にバグがあっても，スーパークラスを修正するだけで済む

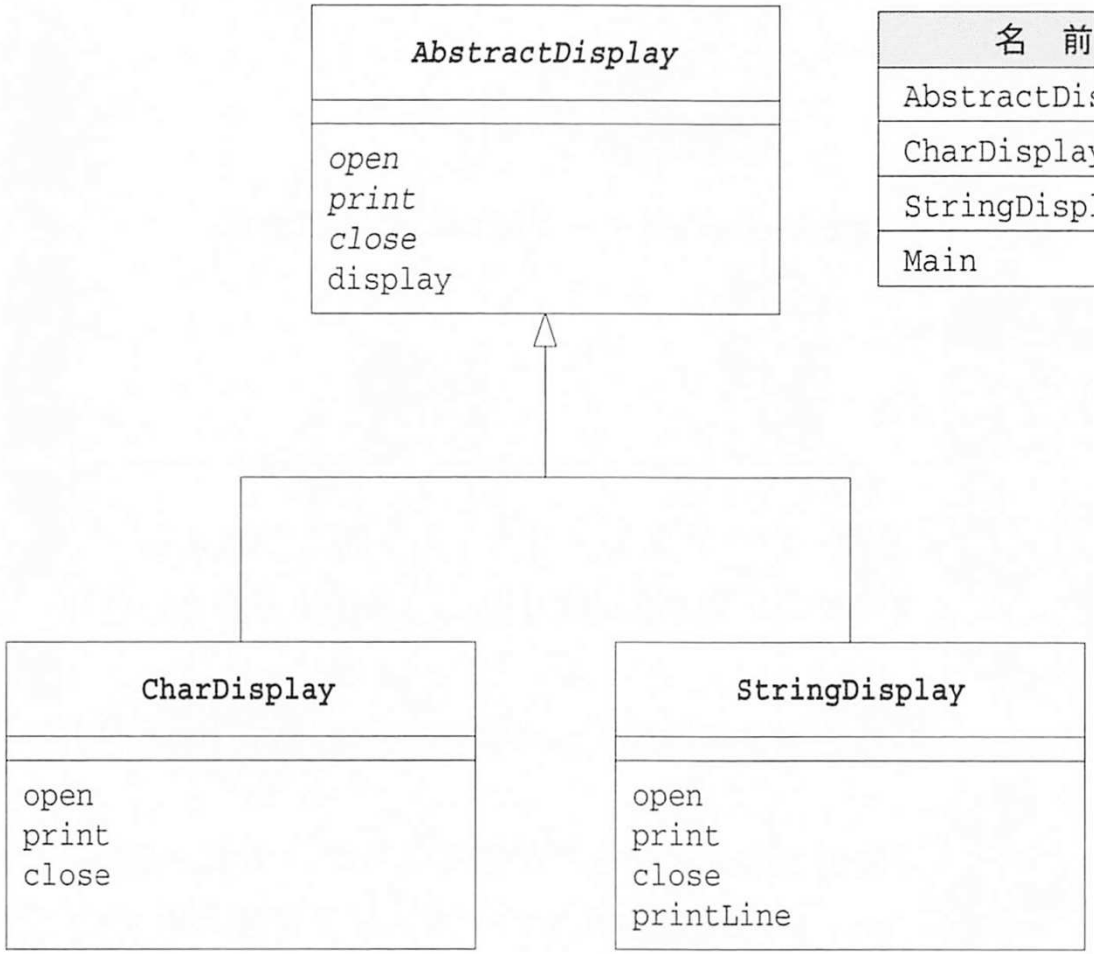
サンプルプログラム

lec02_template_method/*.java

コンパイル方法: javac lec02_template_method/Main.java
実行方法: java lec02_template_method/Main

プログラム内容

- 文字や文字列を5回繰り返して表示する（→大きな流れは共通）
- 文字を出力する場合と，文字列を出力する場合で，出力形式（挙動）を変えたい（→各処理の具体的な内容を変える）



名　前	解　説
AbstractDisplay	メソッドdisplayのみ実装されている抽象クラス
CharDisplay	メソッドopen，print，closeを実装しているクラス
StringDisplay	メソッドopen，print，closeを実装しているクラス
Main	動作テスト用のクラス

実行例（文字の場合）

```
<<HHHHH
```

実行例（文字列の場合）

```
+-------------+
|Hello, world.|
|Hello, world.|
|Hello, world.|
|Hello, world.|
|Hello, world.|
+-------------+
```

結城浩 『Java言語で学ぶデザインパターン入門第３版』 SBクリエイティブ(2021)より

サンプルプログラム

lec02_template_method/*.java

```
public abstract class AbstractDisplay {

    public abstract void open();
    public abstract void print();
    public abstract void close();

    public final void display() {
        open();
        for (int i = 0; i < 5; i++) {
            print();
        }
        close();
    }
}
```

open, print, closedはサブクラスで実装
→ サブクラスで具体的な内容を定義
→ サブクラスごとに実際の動作が異なる

displayメソッド
抽象メソッドの呼び出し方を決めた
テンプレートメソッド
（抽象メソッドを呼び出しを伴う）
→ 処理の大きな流れを定義

結城浩 『Java言語で学ぶデザインパターン入門第３版』 SBクリエイティブ(2021)より

```java
public class StringDisplay extends AbstractDisplay {
    private String string;
    private int width;
    public StringDisplay(String string) {
        this.string = string;
        this.width = string.getBytes().length;
    }
    public void open() {
        printLine();
    }
    public void print() {
        System.out.println("|" + string + "|");
    }
    public void close() {
        printLine();
    }
    private void printLine() {
        System.out.print("+");
        for (int i = 0; i < width; i++) {
            System.out.print("-");
        }
        System.out.println("+");
    }
}
```

String型のstringを持つ

内部処理で必要な関数の定義

それぞれのクラスが,
フィールド変数として表示対象を持つ

AbstractDisplayクラスを継承：
抽象メソッドopen, print, close
を実装する

その一方で,
テンプレートメソッド（display）
の上書きは行わない

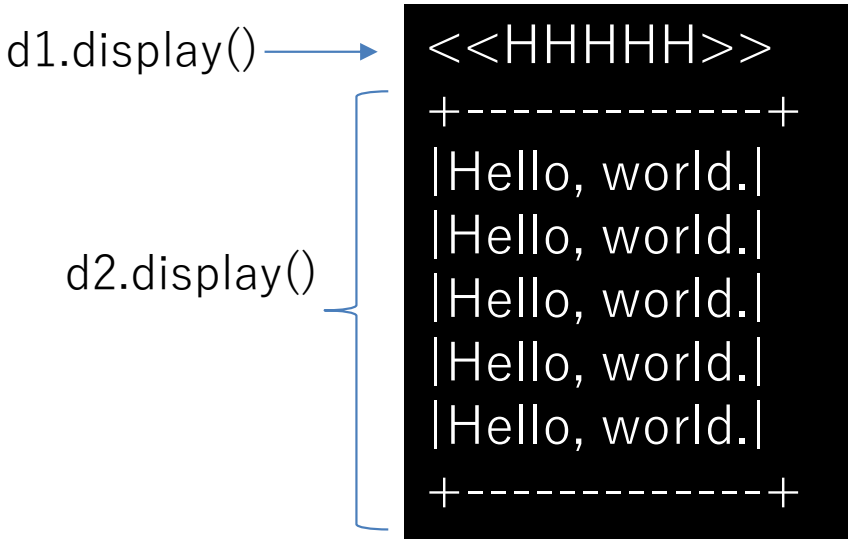
```java
public class CharDisplay extends AbstractDisplay {
    private char ch;
    public CharDisplay(char ch) {
        this.ch = ch;
    }
    public void open() {
        System.out.print("<<");
    }
    public void print() {
        System.out.print(ch);
    }
    public void close() {
        System.out.println(">>");
    }
}
```

char型のchを持つ

```java
public class Main {
    public static void main(String[] args) {
        AbstractDisplay d1 = new CharDisplay('H');
        AbstractDisplay d2 = new StringDisplay("Hello, world.");

        d1.display();
        d2.display();
    }
}
```

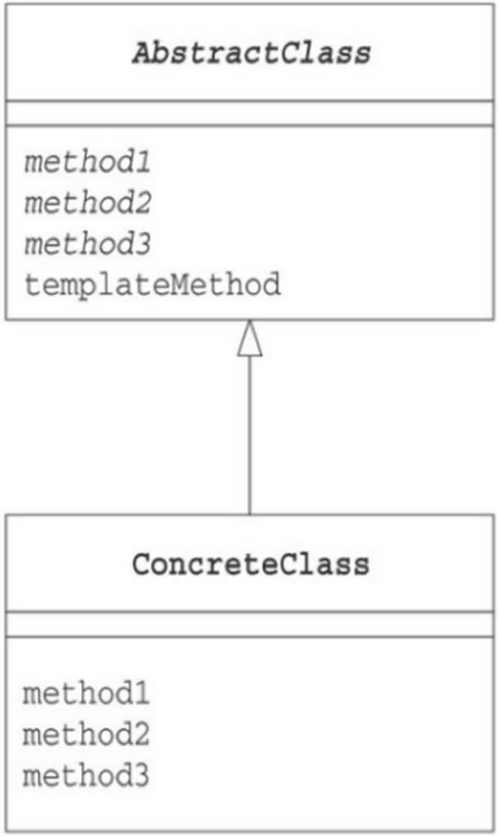
- CharDisplayもStringDisplayもAbstractDisplayを継承している
- AbstractDisplay型のインスタンスd1，d2を生成・実行する
- 具体的な動作（<<>>で囲むかなど）は，サブクラス側で決めている



Template Methodパターンの登場人物

- AbstractClass（抽象クラス）の役
  - テンプレートメソッドを実装する
  - テンプレートメソッドで利用する抽象メソッドを宣言する

- ConcreteClass（具象クラス）の役
  - AbstractClass役のクラスを継承する
  - AbctractClass役で定義されている抽象メソッドを実装する



AbstractDisplay
（テンプレートメソッド + 抽象メソッド）

CharDisplay, StringDisplay
（抽象メソッドを実装）

結城浩 『Java言語で学ぶデザインパターン入門第３版』 SBクリエイティブ(2021)より

考え方を広げるヒント

- （サブクラス間で）ロジックを共通化できる（→ 大きなメリットの一つ）
  - アルゴリズムは，スーパークラスのテンプレートメソッド内で記述する
  - サブクラス側ではアルゴリズムを記述しなくて済む

- スーパークラスとサブクラスの連係プレイ
  - スーパークラスと，サブクラスが綿密の連絡を取って動いている
  - 抽象メソッドを実装する際，そのメソッドがどの様なタイミングで呼び出されるかを理解しておく必要がある
  - テンプレートメソッドのソースレベルでの理解が必要

- サブクラスをスーパークラスと同一視する
  - StringDisplayのインスタンスもCharDisplayのインスタンスも，AbstractDisplay型として扱うことができる.
  - 各インスタンスの具体的なクラスを知る必要はない

演習1

- lec02_template_method/*.java
- 必要最低限の新クラス作成と，Main.javaを修正し，下記のような結果を得られるプログラムを作成せよ.
  - 文字の表示 → CharDisplay.java
  - 文字列の表示 → StringDisplay
  - 整数の表示 → ??
  - 真偽値の表示 → ??
  - Mainはどうする？
  - ※指定ディレクトリ内で作成する
  - ※パッケージ宣言に注意
    - package lec02_template_method;

```
<<HHHHH>>
+-------------+
|Hello, world.|
|Hello, world.|
|Hello, world.|
|Hello, world.|
|Hello, world.|
+-------------+
{ 100 100 100 100 100 }
( true true true true true )
```

- （余力のある学生さん向け）Template Methodパターンを適用したプログラム例を作成せよ

- 指定ファイルの本文に，プログラム内容（テキスト）を貼り付ける（コピペする）形で提出してください

Adapter パターン
～一皮かぶせて再利用～

Adapterパターン

- Adapter（アダプタ）
  - 必要なものと，提供されているものとの間に入って，その間を埋めるもの
  - 異なる複数の機器に接続する際に用いられる中間装置の総称
  - PCのACアダプタ：交流100ボルトを直流12ボルトに変換する
- Adapterパターン：
  - 「すでに提供されているもの」と「必要なもの」の間の「ずれ」を埋めるようなデザインパターン
  - 必要なコード・提供されているコードを変更できない場合がある
    - 例：変更権限がない，変更すると他のコードに影響が出る
  - 典型例
    - 提供されているコード＝外部ライブラリ
    - 必要なコード＝仕様
- ２種類のAdapterパターンが存在する
  1. クラスによるAdapterパターン（継承を使ったもの）
  2. インスタンスによるAdapterパターン（委譲をつかったもの）

Adapterパターン

利用シーン１

- 既に仕様が決まっており，自分には仕様を変更する権限がない
- 仕様と似たことが達成できそうだが，微妙に挙動が異なる外部ライブラリがある

→ 仕様と外部ライブラリの溝を埋めるAdapterプログラムを書けば良い
（外部ライブラリと同様のものをゼロから作成する必要はない）

利用シーン2

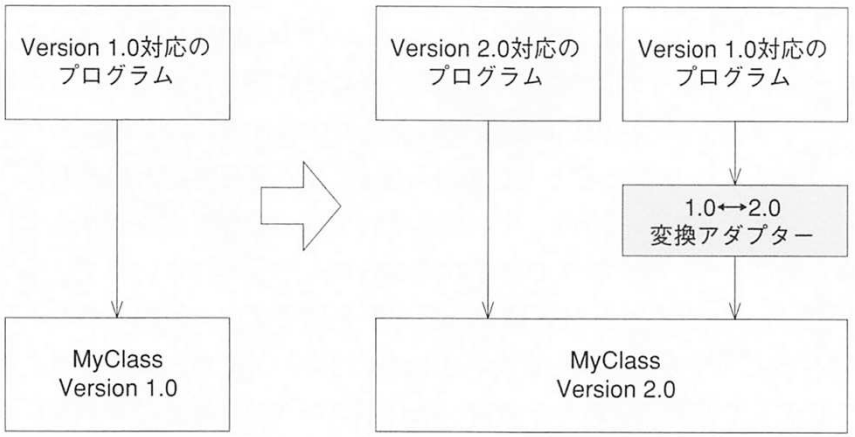
- 既にテストされた巨大な既存プログラムがあり，できればそこに手を入れたくない
- 既存プログラムに新機能を追加するライブラリがあるが，既存プログラムのインタフェースと整合しておらず，そのままライブラリを導入することはできない

→ 既存プログラムとライブラリの溝を埋めるAdapterプログラムを書けば良い
（仮にバグが生じても，原因はAdapterかライブラリにあると絞り込める）

利用シーン3

- システムのバージョンアップを行いたいが，それを行うと旧バージョンのブラウザをサポートできなくなる
- 旧バージョンのブラウザのコードを全部書き直すリソース（金・時間）は無い

→ 新システムと旧バージョンブラウザ関連コードの溝を埋めるAdapterプログラムを書けば良い



結城浩 『Java言語で学ぶデザインパターン入門第３版』 SBクリエイティブ(2021)より

想定シーン

- 必要なもの（仕様）
  - printWeak()で控えめに表示できること
  - printStrong()で強調して表示できること
- 提供されているもの（外部ライブラリ）
  - showWithParen()・showWithAster()を備えたBannerクラス
- 外部ライブラリを活用し，仕様を満たすプログラムを作成するシーンを考える
  - 当然ながら，仕様・外部ライブラリを変更することはできない
  - つまり，仕様と外部ライブラリの溝を埋めるAdapterを作成する必要がある
- やりたいことは，提供されているBannerクラスを再利用して，printWeak(), printStrong()メソッドを持つクラスを作る
  - ゼロベースで printWeak()/printStrong()の内容を作るのではない
  - あくまで，Bannerクラスを利用して作りたい
  - 今回の想定
    - printWeak()での控えめ表示 → Banner#showWithParen()での表示
    - printStrong ()での強調表示 → Banner#showWithAster()での表示

継承を用いたサンプルプログラム

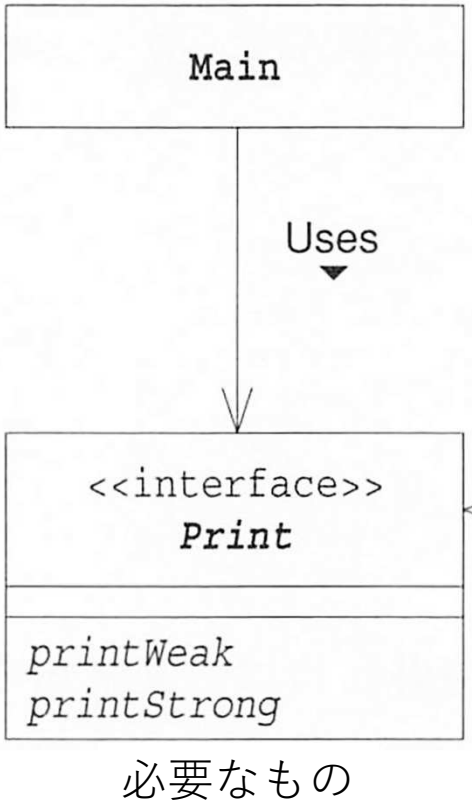
lec02_adapter_inheritance/*.java

```
public class Main {
    public static void main(String[] args) {
        Print p = new PrintBanner("Hello");

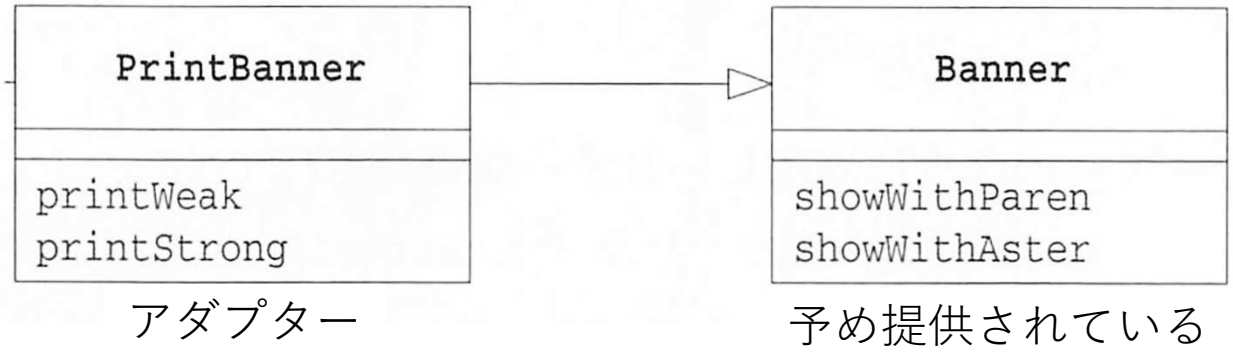
        p.printWeak();
        p.printStrong();
    }
}
```

(Hello)
Hello

コンパイル方法: javac lec02_adapter_inheritance/Main.java
実行方法: java lec02_adapter_inheritance/Main



	電源の比喩	サンプルプログラム
提供されているもの	交流100ボルト	Bannerクラス（showWithParen, showWithAster）
変換装置	アダプター	PrintBannerクラス
必要なもの	直流12ボルト	Printインタフェース（printWeak, printStrong）



結城浩 『Java言語で学ぶデザインパターン入門第３版』 SBクリエイティブ(2021)より

予め提供されているクラス

```
public class Banner {
    private String string;
    public Banner(String string) {
        this.string = string;
    }
    public void showWithParen() {
        System.out.println("(" + string + ")");
    }
    public void showWithAster() {
        System.out.println("*" + string + "*");
    }
}
```

利用したいインタフェース

```
public interface Print {
    public abstract void printWeak();
    public abstract void printStrong();
}
```

継承 (extends) + 実装 (implements) で一皮かぶせる
実装によりインタフェースを揃える
継承により既存機能を利用する

```
public class PrintBanner extends Banner implements Print {
    public PrintBanner(String string) {
        super(string);
    }
    public void printWeak() {
        showWithParen();
    }
    public void printStrong() {
        showWithAster();
    }
}
```

親クラス（Banner）が持つメソッドを利用して
インタフェースを実装する（メソッドの中身を作る）
printWeak()をshowWithParen()に,
printStrong()をshowWithAster()に
読み替えている（適応させている）

継承を使ったAdapterパターン

- Printインタフェース型の変数pに，PrintBannerクラスのインスタンスを代入
  - あくまで p は，Printとして扱われる
  - printWeak, printStrongを使ったプログラムとなる

- Main（使う側）から見ると，Bannerクラスやそのメソッド showWithParen, showWithAsterは隠されている（知る必要すらない）

- Mainから見ると，PrintBannerがどの様に実装されているか（具体的なコード）を知る必要はない
  - インタフェースさえ提供されていれば，PrintBannerの中身を書き換えても，Mainには影響はない

```
public class Main {
    public static void main(String[] args) {
        Print p = new PrintBanner("Hello");

        p.printWeak();
        p.printStrong();
    }
}
```

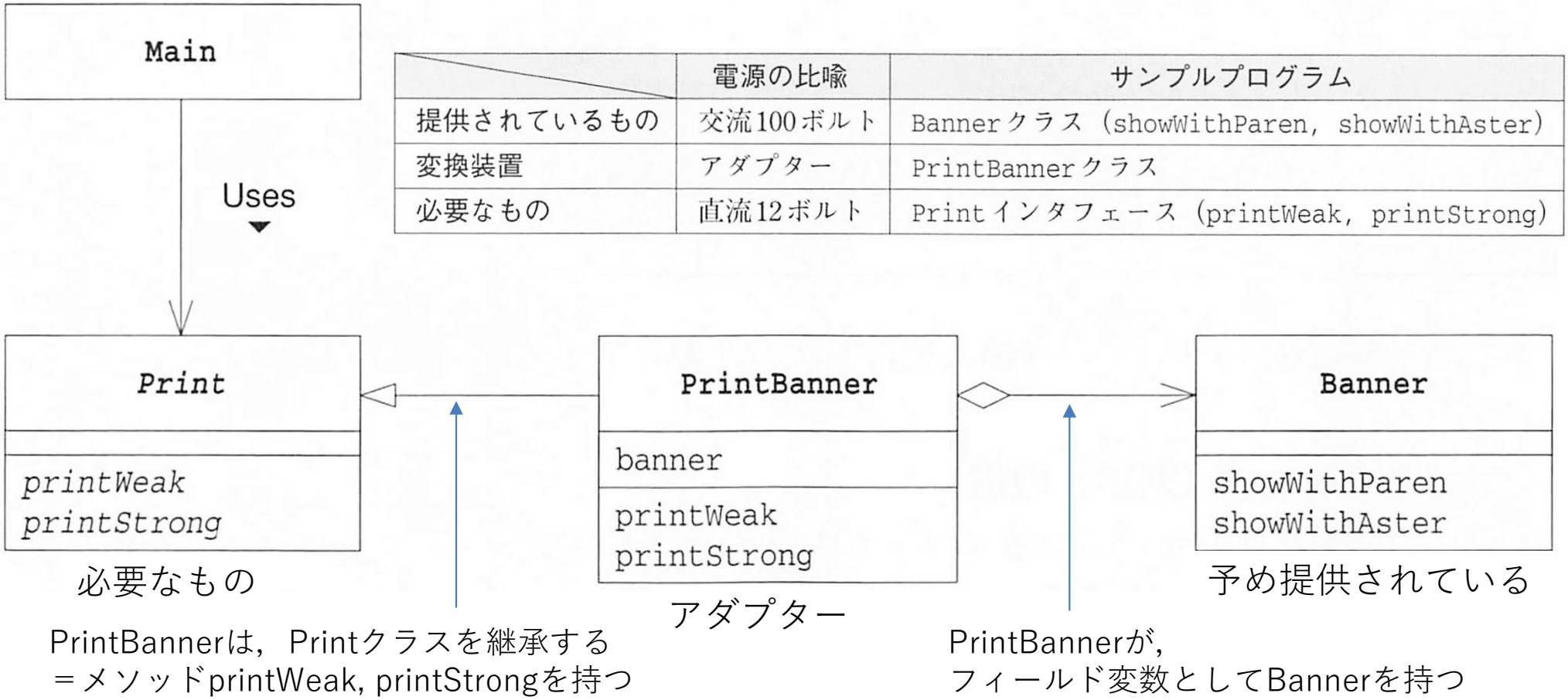
移譲を用いたサンプルプログラム

lec02_adapter_delegate/*.java

- 委譲：「誰かにお任せする」
- 今回は（インタフェースではなく）Printクラス（抽象クラス）を利用
  - Bannerクラスを利用して，Printクラスと同じメソッドを持つクラスを実現する

Main.javaは，
継承を利用した場合と同一

コンパイル方法: javac lec02_adapter_delegate/Main.java
実行方法: java lec02_adapter_delegate/Main



	電源の比喩	サンプルプログラム
提供されているもの	交流100ボルト	Bannerクラス（showWithParen，showWithAster）
変換装置	アダプター	PrintBannerクラス
必要なもの	直流12ボルト	Printインタフェース（printWeak，printStrong）

必要なもの

アダプター

予め提供されている

PrintBannerは，Printクラスを継承する
＝メソッドprintWeak, printStrongを持つ

PrintBannerが，
フィールド変数としてBannerを持つ

結城浩 『Java言語で学ぶデザインパターン入門第３版』 SBクリエイティブ(2021)より

予め提供されているクラス（継承の例と同じ）

```java
public class Banner {
    private String string;
    public Banner(String string) {
        this.string = string;
    }
    public void showWithParen() {
        System.out.println("(" + string + ")");
    }
    public void showWithAster() {
        System.out.println("*" + string + "*");
    }
}
```

利用したい抽象クラス

```java
public abstract class Print {
    public abstract void printWeak();
    public abstract void printStrong();
}
```

```java
public class PrintBanner extends Print {

    private Banner banner;

    public PrintBanner(String string) {
        this.banner = new Banner(string);
    }

    public void printWeak() {
        banner.showWithParen();
    }
    public void printStrong() {
        banner.showWithAster();
    }
}
```

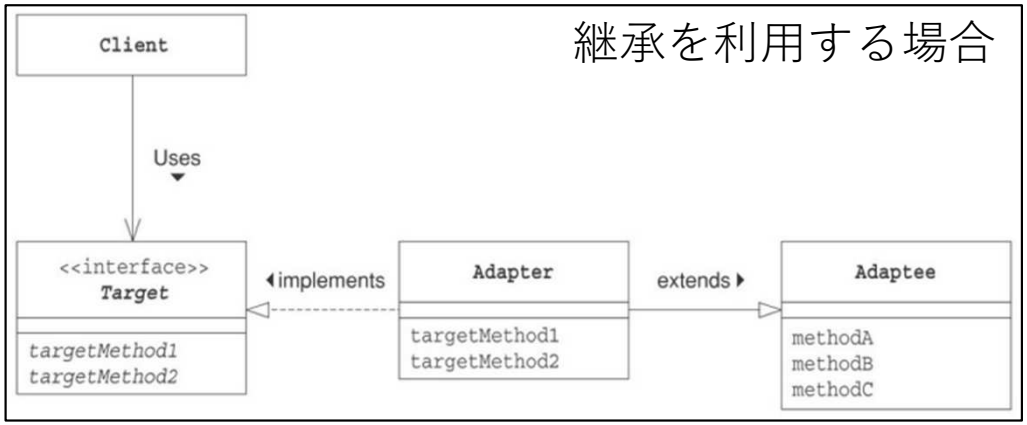
継承 (extends) + 委譲で一皮かぶせる
継承により型を揃える（メソッド名も揃う）
各メソッドの実現は，bannerに委譲する

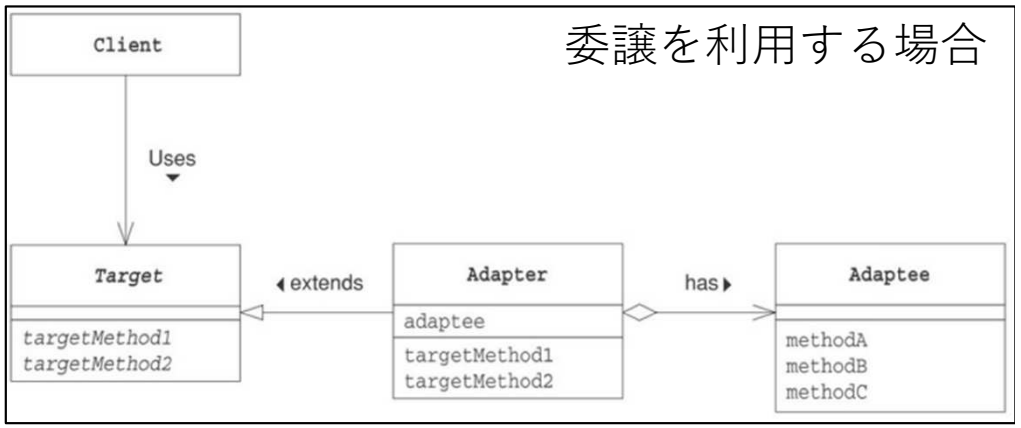
Bannerクラスのインスタンスを生成・保持する

bannerインスタンスが持つメソッドを利用して
　インタフェースを実装する（メソッドの中身を作る）
Bannerインスタンスの適当なメソッドを呼びだし
printWeak()をshowWithParen()に，
printStrong()をshowWithAster()に
読み替えている（適応させている）

Adapterパターンの登場人物

- Target（対象）の役
  - いま必要となっているメソッドを定めている役
- Client（依頼者）の役
  - Target役のメソッドを使って仕事をする役
- Adaptee（適合される側）の役
  - 既に用意されているメソッドを持っている役
- Adapterの役
  - Adapterパターンの目的：Adaptee役のメソッドを使ってTarget役を満たす
  - クラスによるAdapterパターン：「継承」を使ってAdaptee役を利用
  - インスタンスによるAdapterパターン：「委譲」を使ってAdaptee役を利用





結城浩 『Java言語で学ぶデザインパターン入門第３版』 SBクリエイティブ(2021)より

演習2

- lec02_adapter/*.java
- クラスEllipseは，平面座標上の楕円を表すクラスである．このクラスを利用して，Shapeインタフェースを実装した「円」を表すクラスCircleを作成せよ．

  Ellipseは楕円の中心座標に着目してメソッドが構成されている．
  Shapeは図形の左上座標に着目してメソッドが構成されている．
  両者のギャップを埋めるのがアダプタであるCircleの役割

  - ※指定ディレクトリ内で作成する
  - ※パッケージ宣言に注意
    - package lec02_adapter;

- （余力のある学生さん向け）Adapterパターンを適用したプログラム例を作成せよ

- 指定ファイルの本文に，プログラム内容（テキスト）を貼り付ける（コピペする）形で提出してください

Iteratorパターン
～1つ1つ数え上げる～

Iteratorパターン

- 配列要素をすべて表示するプログラム例
  - for(int i = 0; i < arr.length; i++){ System.out.println(arr[i]); }
  - ループ変数 i に着目
    - 変数 i は，最初0で初期化され，1, 2, 3…と増えていく
    - その都度，arr[i]を出力する
  - iの値を1増加させることで，着目している要素を「次」「その次」に進める
  - 結果として，配列arrの全要素を”走査”することになる

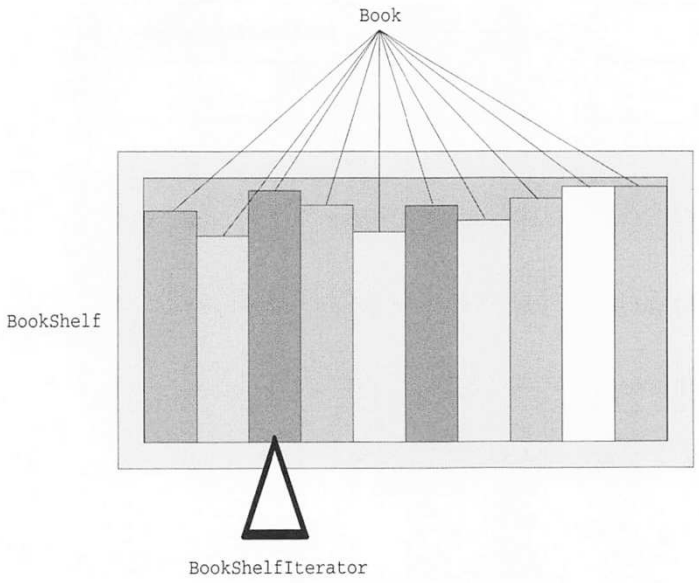
- Iteratorパターン：変数 i の働きを抽象化し一般化する
  - 配列（集合）要素を，順番に指し示し，全体を走査する
  - Iterator：反復子．上記の役割を果たすインタフェース

- 何かがたくさん集まっているとき，各要素を順番に指し示していく処理を方法を統一するための考え方
- 利用シーン
  - 集合体を走査したいが，集合体の表現が変わる可能性がある場合
  - 例えば，配列かもしれないし，ArrayListかもしれないし，未知のデータ構造かもしれない

Iteratorパターン

結城浩 『Java言語で学ぶデザインパターン入門第３版』
SBクリエイティブ(2021)より

- Bookの集合を含むBookShelfを走査する
- Iteratorパターンを用いない場合の問題点
  - BookShelfの具体的な実装方法
    （例：配列，ArrayList）を把握しないと
    Mainクラスが書けない
  - BookShelfの実装方法が変わるたびに
    Mainクラスを修正しないといけない




```
public void main(String[]args) {

    Book[] bookShelf = new Book[4];
    bookShelf[0] = new Book("Animal Farm");
    bookShelf[1] = new Book("Bible");
    bookShelf[2] = new Book("Cinderella");
    bookShelf[3] = new Book("Đaddy-Long-Legs");

    for(int i = 0; i < bookShelf.length; i++) {
        Book book = bookShelf[i];
        System.out.println(book.getName());
    }
}
```

bookShelfが配列として実装されている場合

```
public void main(String[]args) {

    List<Book> bookShelf = new ArrayList<Book>();
    bookShelf.add( new Book("Animal Farm") );
    bookShelf.add( new Book("Bible") );
    bookShelf.add( new Book("Cinderella") );
    bookShelf.add( new Book("Đaddy-Long-Legs") );

    for(int i = 0; i < bookShelf.size(); i++) {
        Book book = bookShelf.get(i);
        System.out.println(book.getName());
    }
}
```

bookShelfがリストとして実装されている場合

両者の間で，データ走査方法（データへのアクセス方法）が異なる点に注意 → これを統一したい

サンプルプログラム

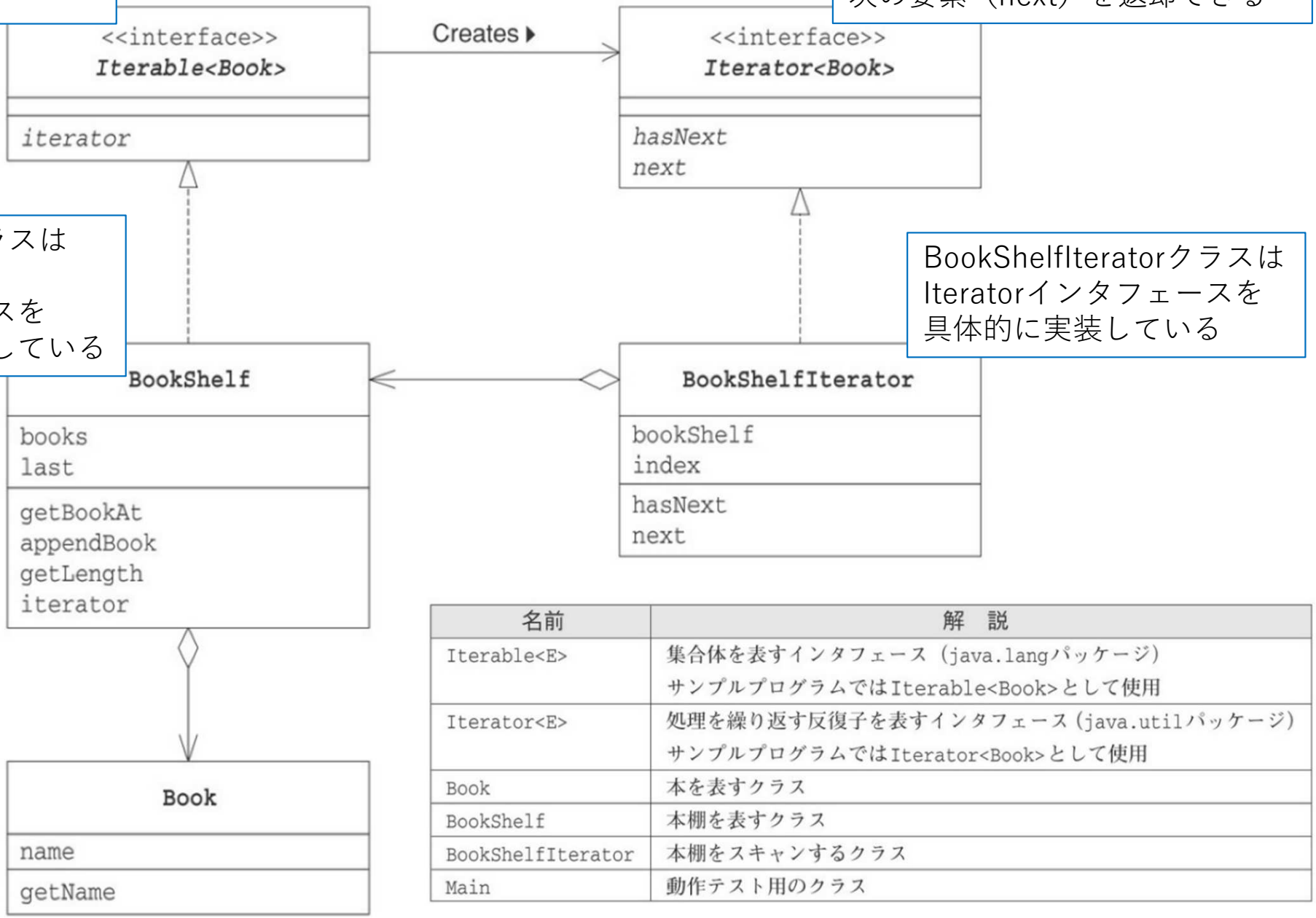
lec02_iterator/*.java

Aggregate（集合体）は数え上げの道具を返却できる。

Iterator（数え上げの道具）は集合体を走査して次の要素の有無（hasNext）と，次の要素（next）を返却できる

BookShelfクラスはAggregateインタフェースを具体的に実装している

BookShelfIteratorクラスはIteratorインタフェースを具体的に実装している



名前	解　説
Iterable<E>	集合体を表すインタフェース（java.langパッケージ） サンプルプログラムではIterable<Book>として使用
Iterator<E>	処理を繰り返す反復子を表すインタフェース（java.utilパッケージ） サンプルプログラムではIterator<Book>として使用
Book	本を表すクラス
BookShelf	本棚を表すクラス
BookShelfIterator	本棚をスキャンするクラス
Main	動作テスト用のクラス

結城浩 『Java言語で学ぶデザインパターン入門第３版』 SBクリエイティブ(2021)より

lec02_iterator/*.java

具体的な実装を意識していない

java.lang.iterableより

```
public interface Iterable<E> {
    public abstract Iterator<E> iterator();
}
```

Creates ▶

java.util.Iteratorより

```
public interface Iterator<E> {
    public abstract boolean hasNext();
    public abstract E next();
}
```

「次の要素」が存在するか？

「次の要素」（T型）を返す
& 内部状態を次に進める

実装

実装

内部では配列を利用してBookインスタンス集合を管理
外向けにはメソッドを公開する

```
public class BookShelf
                implements Iterable<Book> {

    private Book[] books;
    private int last = 0;
    public BookShelf(int maxsize) {
        this.books = new Book[maxsize];
    }
    public Book getBookAt(int index) {
        return books[index];
    }
    public void appendBook(Book book) {
        this.books[last] = book;
        last++;
    }
    public int getLength() {
        return last;
    }
    public Iterator<Book> iterator() {
        return new BookShelfIterator(this);
    }
}
```

自身を渡す

Book型の反復子を返すメソッドを実装

```
public class BookShelfIterator
                    implements Iterator<Book> {
    private BookShelf bookShelf;
    private int index;
    public BookShelfIterator(BookShelf bookShelf) {
        this.bookShelf = bookShelf;
        this.index = 0;
    }
    public boolean hasNext() {
        if (index < bookShelf.getLength()) {
            return true;
        } else {
            return false;
        }
    }
    public Book next() {
        if (!hasNext()) {
            throw new NoSuchElementException();
        }
        Book book = bookShelf.getBookAt(index);
        index++;
        return book;
    }
}
```

index =
走査位置を表す変数
で状態を管理

エラーを投げる：未学習内容

走査対象bookShelfを持つがその実装は「知らない」
長さ(getLength)と要素獲得（getBookAt）のみ利用

lec02_iterator/*.java

コンパイル方法: javac lec02_iterator/Main.java
実行方法: java lec02_ iterator /Main

```java
public class Main {
    public static void main(String[] args) {

        BookShelf bookShelf = new BookShelf(4);
        bookShelf.appendBook( new Book("Around the World in 80 Days") );
        bookShelf.appendBook( new Book("Bible") );
        bookShelf.appendBook( new Book("Cinderella") );
        bookShelf.appendBook(new Book("Daddy-Long-Legs"));

        Iterator<Book> it = bookShelf.iterator();
        while (it.hasNext()) {
            Book book = it.next();
            System.out.println(book.getName());
        }
    }
}
```

BookShelf型の
インスタンスを生成

Bookのインスタンス
を追加

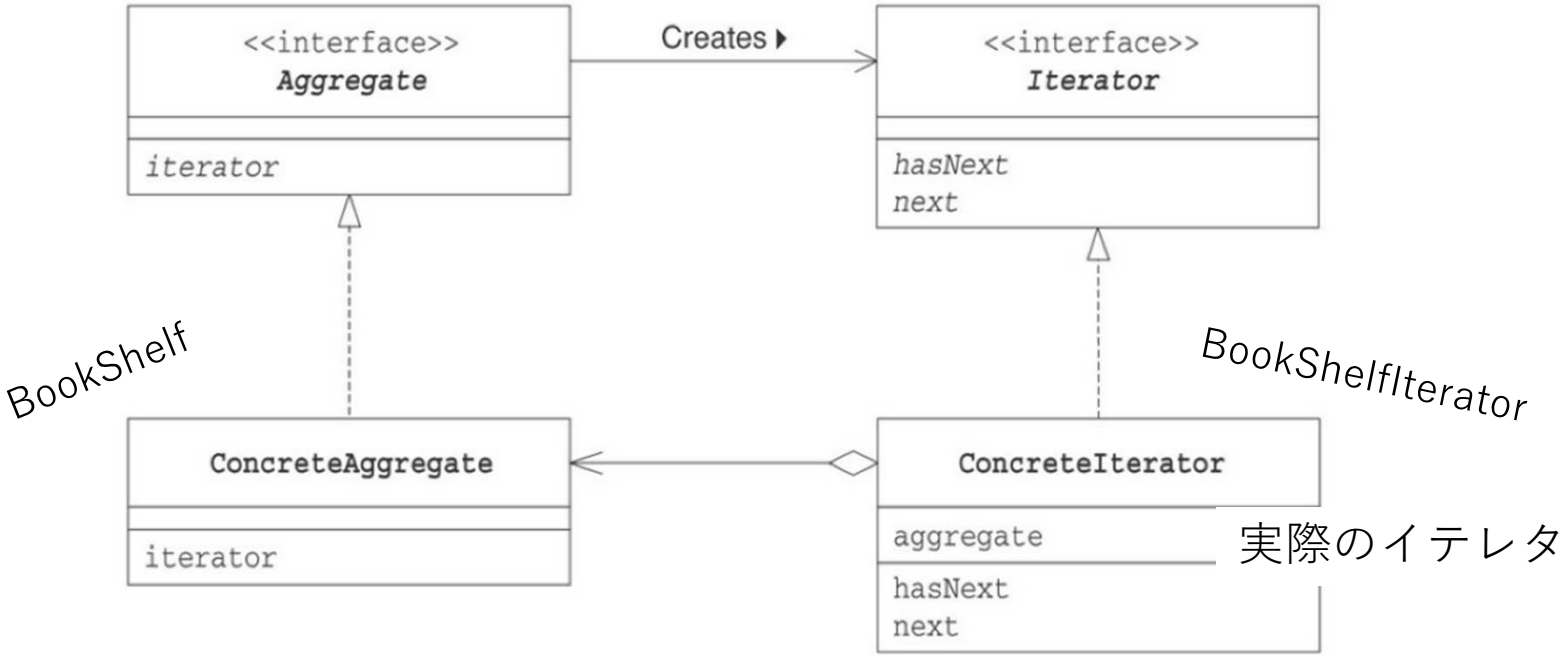
Iteratorの獲得と走査
　hasNextが真の間は繰り返す
BookShelfの具体的な実装は意識していない

```java
public class Book {
    private String name;
    public Book(String name) {
        this.name = name;
    }
    public String getName() {
        return name;
    }
}
```

```
Around the World in 80 Days
Bible
Cinderella
Daddy-Long-Legs
```

Iteratorパターンの登場人物

- Iteratorの役
  - 要素を順番にスキャンするインタフェース（hasNext, next）を定義
- ConcreteIteratorの役
  - Iterator役のインタフェースを実際に実装するクラス
- Aggregateの役
  - Iterator役を生成するメソッドを定義
  - 「私が持っている要素を順番にスキャンしてくる人」を作りメソッドを持つ
- ConcreteAggregateの役
  - Aggregate役のインタフェースを実装したクラス
  - ConcreteIteratorのインスタンスを生成する



結城浩 『Java言語で学ぶデザインパターン入門第３版』 SBクリエイティブ(2021)より

考えを広げるためのヒント

- 実装がどうであれ，Iteratorを使える
  - 配列ならfor文で回せばよい？Listなら拡張for文？
  - Iteratorを利用することで，実装とは切り離して数え上げを行う
    - 右のプログラムで利用されているのは，Iteratorのメソッド（hasNext,Next）だけBookShelfの実装に依存しない

```
while( it.hasNext() ){
    System.out.println( it.next() );
}
```

  - BookShelfを実装した人が，配列をやめてListを利用した場合を考える
    - while文を利用しているプログラム（Main.java）は変更の必要がない
    - 既存プログラム（Main.java）を変えずに，BookShelfの実装変更が可能

- AggregateとIteratorの対応は必要
  - ConcreteIterator役は，ConcreteAggregateの実装に依存

考えを広げるためのヒント

- 「次」は間違いやすい
  - Nextメソッド：「次の要素を返す」＋「内部状態の変更」
    - 現在の要素を返しつつ，次の位置に進める
  - return_Current_Element_and_Advance_to_Next_Position の意味
- 「最後」も間違いやすい
  - hasNextは，最後の要素を得る前はtrue/最後の要素を得た後はfalseを返す
  - hasNextは「次にnextメソッドを呼び出しても大丈夫か？」の意味
- 複数のIterator
  - 数え上げの仕組みがAggregate役の外に置かれている
  - 一つのConcreteAggregate役に対し，複数のConcreteIteratorを作成可能
- イテレータの種類いろいろ
  - サンプルでは，順方向に一度だけ走査を行ったが，他にも考えられる
    - 最後尾から開始して，逆方向に進む
    - 順方向にも逆方向にも進める（ hasPrevious, previousメソッド）
    - 番号を指定して， そこにジャンプするなど

演習3

- lec02_iterator/*.java
- BookShelfは，配列を用いて Bookの集合を管理している．このクラスを，リストを用いてBook集合を管理するに変更しなさい
  - BookShelfの各API（メソッドの型，名前，引数）は変更しないように！
    - APIの変更なし＝外部に対する影響なし
  - ※指定ディレクトリ内で作成する
  - ※パッケージ宣言に注意
    - package lec02_iterator;
- （余力のある学生さん向け）BookShelfに対し，それぞれ以下のイテレータを作成しなさい
  - 含まれる要素を「逆順に走査する」
    Iterator<Book>を実装したクラスReverseIteratorを作成しなさい
  - 含まれる要素を「（前から後ろに）繰り返し3回走査する」
    Iterator<Book>を実装したクラスThreeTimesIteratorを作成しなさい
- （余力のある学生さん向け）Iteratorパターンを適用したプログラム例を作成せよ
- 指定ファイルの本文に，プログラム内容（テキスト）を貼り付ける（コピペする）形で提出してください

参考：ListインタフェースとIteratorインタフェース

- java.util.List<T>インタフェース：T型のインスタンスを含むリスト
- java.util.Iterator<T> インタフェース：T型のコレクションに対する反復子
  - boolean hasNext()：次の要素があるかを返す
  - T next()：次の要素を返す

```
import java.util.Arrays;
import java.util.Iterator;
import java.util.List;

public class ListExample {

    public static void main(String[]args){

        List<String> strList = Arrays.asList( new String[]{"apple", "orange", "grape", "mango"} );

        for(int i = 0; i < strList.size(); i++){
            System.out.println( strList.get(i) );
        }

        Iterator<String> itStr = strList.iterator();
        while( itStr.hasNext() ){
            String str = itStr.next(); while文を用いた
            System.out.println(str);        繰り返しの例
        }

        List<Integer> intList = Arrays.asList( new Integer[]{ 1, 2 ,3, 4 } );

        for(Iterator<Integer> itInt = intList.iterator(); itInt.hasNext(); ){
            int v = itInt.next();
            System.out.println(v);                    for文を用いた繰り返しの例
        }
    }
}
```

格納するインスタンスの型を指定する

Arrays.asList (配列)メソッドで,
配列要素を含むリストインスタンスを生成

リスト.get(i) で，リストのi番目の要素を獲得

Iteratorインタフェースを実装した
インスタンスを返すメソッド

参考：Listインタフェースと Iteratorインタフェース

```
public class Ball {
    int x;
    int y;
    int r;
    Ball(int x, int y, int r){
        this.x = x;
        this.y = y;
        this.r = r;
    }
    public String toString(){
        return "("+x+","+y+","+","+r+")";
    }

    public static void main(String[]args){
        List<Ball> ballList = Arrays.asList(new Ball[]{ new Ball(1,1,1), new Ball(2,2,2) });

        Iterator<Ball> it = ballList.iterator();
        while( it.hasNext() ){
            Ball ball = it.next();
            System.out.println( ball );
        }

        ballList.add( new Ball(3,3,3) );
        for(Iterator<Ball> it2 = ballList.iterator(); it2.hasNext(); ){
            System.out.println( it.next() );
        }
    }
}
```

List<Ball> ballList : Ballクラスのインスタンスを格納するリスト
java.util.ArraysのasListメソッドで,
　Ballクラスを格納した配列から，リストを生成する

ballListに対するイテレタの獲得

次の要素があるか？の判定

次の要素（Ball型のインスタンス）の獲得

addメソッド：要素の追加